Executable Conformance for a Secure-Multiparty Application: What Implementation-Level Evidence Can Certify, and Where It Stops Short of a Privacy Proof

First draft — artifact baseline commit `305a3a8`

Harshit Kargeti*

August 3, 2026

Abstract

Secure-multiparty computation (MPC) *applications*—programs written on top of an MPC framework—increasingly ship with continuous-integration checks that are easy to mistake for privacy guarantees. We report a methodology and a boundary finding for one such application: the deterministic-query fragment of the knowledge-threshold belief computation of Mardziel, Hicks, Katz and Srivatsa (PLAS 2012), implemented in MP-SPDZ. Our first contribution is an *executable-conformance* methodology: an independently written plaintext oracle, a tiny hand-verifiable fixture containing both an accepted and a rejected invocation, an anti-echo interface contract that keeps expected outputs out of the circuit, and SHA-bound, recomputed raw evidence. This makes a green check mean that a *defined* functionality agrees with an *independent* reference on the inputs tested, rather than that two co-derived models agree. Our second contribution is a negative, boundary result. We attempted to *certify*, by static inspection of the compiled MP-SPDZ assembly, that each recipient's verdict is delivered privately. Six escalating adversarial re-reviews each produced a concrete build that leaked a verdict yet passed the then-current gate; the sixth cannot be closed, because the secret is routed into a comparison subtape through the very register-argument channel the honest code uses, and because a masked comparison-open and a raw secret-reveal are the same opcode. We therefore scope the static check honestly as a *linter* and articulate the hand-off: a real non-leakage guarantee needs privacy of the executed circuit *and* a semantic source-to-spec binding, neither of which functional conformance supplies in general. We are careful not to over-claim: we show the *studied syntactic/opcode-level approach* is unsound against a source-controlling author, and state a stronger impossibility only as a conjecture with its missing proof obligations named. The artifact is public and reproducible against a pinned backend.

1 Introduction

An MPC framework such as MP-SPDZ [2] compiles a high-level program into a protocol that several parties execute so that each learns only its prescribed output. Building a correct *application* on top of such a framework is a separate problem from the framework's own security proof: the application

*Author list, affiliations, and acknowledgments are **[OPEN: author metadata]**. This manuscript was produced through a three-party adversarial loop (direction, implementation, adversarial review) described in Section 6; the division of credit is not yet fixed.

author can, by accident or by intent, write source that reveals a secret the functionality was meant to protect. When such an application is wired into continuous integration, a green check is tempting to read as evidence that the privacy property holds. This paper is about what that check can and cannot establish.

Our vehicle is the knowledge-threshold belief computation proposed by Mardziel, Hicks, Katz and Srivatsa at PLAS 2012 [1]. In their setting, N parties each hold a secret; before releasing a query's output, the parties check *inside* the MPC whether releasing it would let any party's posterior belief about another party's secret exceed a threshold, and—crucially—whether a party receives output or a rejection must not itself be observable by the others. The authors implemented one of their two mechanisms and *simulated* the other, "SMC belief tracking," without implementing it. We build the smallest faithful circuit for a deterministic-query fragment of that second mechanism, not as an end in itself, but as a concrete artifact on which to study conformance and its limits.

We are explicit about what this paper is *not*. It is not "the first implementation of PLAS belief tracking," and it makes no such claim: an earlier round of adversarial review on this project established, correctly, that implementing and timing a 2012 construction is not by itself a research contribution absent a new protocol, theorem, compiler, security result, or empirically supported systems insight.[docs/review-log.md R-10] The contributions here are methodological and boundary-finding, and the artifact is their vehicle.

Contributions.

1. **An executable-conformance methodology for an MPC application** (Section 3): an independent oracle, an accept-and-reject fixture, an anti-echo interface, and recomputed SHA-bound evidence, so that CI checks a defined functionality against an independent reference.
2. **A reproducible conformance artifact** for the deterministic-query fragment of `threshold_SMC` (N=3, domain $\{0, 1, 2\}$, thresholds 1/2), with 92 conformance tests, a 228-case adversarial coverage sweep, and five committed executable negative controls, all green against a pinned backend (Section 3.1).
3. **An empirical boundary result with a taxonomy** (Section 4): six escalating static-delivery bypasses plus an independent semantic bypass, each reproduced end to end, showing that the studied opcode-level certification approach is unsound against a source-controlling author, and isolating the two structural reasons.
4. **A precise articulation of the hand-off** (Section 5): the two separate properties a real non-leakage guarantee needs, and why conformance supplies neither in general.
5. **The adversarial review process as a reusable discipline** (Section 6), with an append-only, auditable log including a self-refutation.

We foreground the honest scope of the boundary result now, so it is not over-read. What we *demonstrate* is empirical: six concrete builds and a structural argument. What we do *not* assert is a general impossibility theorem for static analysis; Section 4.4 states the stronger claim only as a conjecture and lists what a proof would require. A dataflow-aware analysis is explicitly not conjectured impossible—indeed it is one of the routes to a real guarantee.

2 Background

The functionality. Following PLAS 2012 [1], N parties hold secrets $s_1, \ldots, s_N$ in a finite domain D. Each observer j maintains a belief δ_j, a distribution over the other parties' secrets. A knowledge-

threshold policy for party i with public threshold t_i demands that no other party ever assign posterior probability greater than t_i to any single value of s_i. For a query Q, the mechanism runs, per recipient j, a check `tcheck` that quantifies over *every possible output* o of Q: it revises δ_j by conditioning on $Q(s)=o$ and rejects if any protected party's marginal would exceed its threshold. Quantifying over all possible outputs—not merely the actual one—is what makes the accept/reject decision simulatable and stops the rejection itself from leaking.[conformance/CONTRACT.md; Fig. 4] Acceptance uses $\leq t_i$ (equality allowed); the boundary is strict $>$.

We study the fragment in which queries are *deterministic*, total, and public, and their choice is independent of the secrets. Two queries instantiate it: `p1_is_max` ($x_0 \geq x_1 \wedge x_0 \geq x_2$) and `sum_even` ($x_0{+}x_1{+}x_2$ even). Probabilistic queries need likelihood weighting and are out of scope.[conformance/oracle.py; CONTRACT.md]

Private delivery. On accept, recipient P_j learns the actual output o; on reject it learns a rejection, and its belief is unchanged. Each recipient's pair $(\text{accept}_j, \text{payload}_j)$ must be delivered privately, with the payload masked to a fixed constant on reject, and no party may learn another party's verdict.[conformance/INTERFACE.md]

The backend. Circuits are compiled and run with MP-SPDZ pinned at commit `9d809599...`, using `replicated-ring-party.x`: three-party replicated secret sharing (Rep3) over $\mathbb{Z}_{2^{64}}$, semi-honest, honest-majority.[.github/workflows/benchmark.yml; ADVERSARY.md] The MP-SPDZ compiler emits a manifest of *tapes* (a main tape plus comparison subtapes for equality/less-than), each a sequence of opcodes. Two facts about this compiled form drive Section 4: a public reveal and a legitimate masked comparison-open are the *same* opcode (`asm_open`/`vasm_open`), and a value computed in one tape reaches another only through memory or through register arguments passed by `call_tape`/`call_arg`—a cross-tape register reference is a compiler error.

Threat model and scope box. We assume at most one semi-honest corruption, no collusion, authenticated encrypted channels, and process isolation. The test harness, however, runs all three parties as one operating-system user on one host: the isolation assumption is stated, not enforced by the test.[conformance/ADVERSARY.md] We make no claim under collusion, a malicious party, unencrypted channels, or a shared-host adversary, and no performance claim of any kind.

3 Method: Executable Conformance for an MPC Application

The discipline has four parts, each of which exists to defeat a specific way a conformance check can be vacuous.

An independent oracle. The plaintext oracle is written from the paper, independently of the circuit and of the project's earlier exact-rational reference model, and transcribes the all-possible-outputs `tcheck` semantics directly.[conformance/oracle.py] Independence matters: comparing the circuit against a reference co-derived from it would only confirm that two models built from the same misunderstanding agree. The transcription is not assumed correct—it carried a fatal error (conditioning on the actual output only, rather than all outputs) that adversarial review caught and that we corrected, adding a discriminating regression test whose two semantics give different visible results.[docs/review-log.md R-13; test_conformance.py::test_discriminating_case]

An accept-and-reject fixture. The fixture uses three parties, domain $\{0, 1, 2\}$, thresholds $1/2$, and two invocations chosen so that one invocation accepts for all recipients and a later one rejects for one recipient while the others accept, exercising per-recipient divergence and—the property the earlier circuits violated—belief preservation on reject.[conformance/CONTRACT.md] A single fixture is a regression vector, not a proof of general conformance; we say so, and add targeted tests for the equality boundary, an unrealized output, an impossible event, a non-uniform prior, and the threshold range.

An anti-echo interface. Before any circuit was written, the interface was fixed to forbid an "echo" circuit that simply returns what it is told.[conformance/INTERFACE.md; docs/review-log.md R-17,R-18] The expected outputs and expected post-state never enter the circuit—neither compiled in nor as runtime inputs; they exist only in the external harness, which obtains them independently from the oracle. Circuit inputs are exactly the public parameters (N, D, Q, t_i) and the secret-shared secrets and beliefs. The harness runs at least one secret state beyond the fixture, so a constant-output circuit cannot pass by accident. Beliefs travel as dense, unnormalized non-negative integer weight vectors in a fixed lexicographic order; two states are equal iff their weight vectors are proportional. The threshold check avoids division: for public $t_i = a/b$, marginal M, and total Z, compare $b\,M \leq a\,Z$. Because MP-SPDZ comparison is *signed*, the no-wraparound requirement is $B < 2^{k-1}$ with $B = \max(a, b) \cdot S \cdot W$, not the naive $2^k > B$; the naive bound is wrong and we pin a counterexample.[INTERFACE.md; circuit_spec.py; R-22] For the fixture, $B = 54 \ll 2^{63}$.

Recomputed, SHA-bound evidence. Every run retains a typed JSONL record: each party's raw stdout/stderr, return code, launch command, a source hash, a delivery signature, provenance, and channel status.[conformance/validate_evidence.py; private_run.py] The validator enforces an exact field set and types, re-parses each retained transcript, and, in CI under `--recompute`, requires each record's `source_sha256` and delivery signature to equal values recomputed independently from the checked-out source and a fresh compile. It also binds each record to a canonical case by recomputing the input hash and the oracle verdict and requiring a bijection, so a replayed duplicate, an unbound input, or a forged verdict value is rejected. Provenance is real: committed local evidence is labeled unbound (`bound:false`); only CI runs carrying `GITHUB_SHA` are bound, and the job asserts the MP-SPDZ head equals the pin.[R-34,R-37,R-43; benchmark.yml] A dedicated `pipefail` canary proves a failing suite cannot go green through a logging pipe.[R-31]

3.1 Artifact and results

The methodology is instantiated as a public artifact against the pinned backend. Functional conformance holds on the fixture (two invocations, accept then reject) and two additional valid states, matching the oracle on verdicts, payloads, and reconstructed weights (4/4). The adversarial coverage sweep passes 228 of 228 cases (216 single-invocation and 12 carried), spanning all 27 secrets, both queries, and uniform, non-uniform, and per-party-scaled integer weights near the bit bound.[coverage.py; results-coverage.txt] The test suites—92 conformance tests (46 functional, 46 private-gate) and 9 reference tests—are green, as is the CI job that builds MP-SPDZ at the pin, asserts head equals pin, runs the harness, coverage, and private-delivery gate, and validates the uploaded evidence under `--require-bound --recompute`. The five committed negative controls of Section 4 are rejected by the linter. No timing is reported: the repository's older timings measure a different, non-conforming circuit and are explicitly not to be cited.

4 The Private-Delivery Gate and Its Adversarial Refutation

Functional conformance says nothing about *privacy* of delivery: a circuit can compute the right verdicts and then broadcast them. We therefore attempted a separate, automatic gate that a build must pass before it can be called private, and subjected it to adversarial re-review. The gate has three layers: a static inspection of the compiled delivery, a strict runtime transcript check, and the recomputed evidence binding of Section 3. Layers two and three are sound for what they check. Layer one is the subject of this section.

4.1 Six escalating bypasses

The motivating gross case is a build whose `reveal_to(j)` is changed to a public `reveal()` while keeping the per-party print: it compiles to a public open of all six verdicts, yet a stdout oracle reports success, because a stdout check cannot distinguish private output from a public open followed by private printing.[docs/review-log.md R-41] That moved us from a stdout oracle to inspecting the compiled delivery, and it is committed as the negative control `threshold_smc_leaky.mpc`. Six further re-reviews followed, each a concrete build that leaked a verdict yet passed the gate that existed when it was found. Table 1 is the taxonomy.

Each closure sharpened the gate: from main-tape-only to whole-manifest inspection (B1); from a name-based allowlist to content-based rules (B2); from trusting the open flag to pinning the exact multiset of comparison subtapes (B3); from names and counts to forbidding the memory channel a verdict would need to enter a subtape (B4); and from a scalar-only memory pattern to one covering vectorized and gf2n forms (B5).[review-log.md R-42–R-46] The clean build stores nothing in memory in its main tape and touches no memory in its subtapes, so forbidding memory is a sound restriction with no false positives: memory was an attacker-introduced channel.

4.2 Why this is structural, not a patch away

B6 does not fit the pattern. The verdict is a register in the main tape; a cross-tape register reference is a compiler error; so the only ways a verdict can reach a subtape's open are memory—now forbidden—or the register arguments passed by `call_tape` and received by `call_arg`. But that register channel is exactly how the honest equality and less-than subtapes receive their operands. Forbidding it would reject the honest build. So B6 cannot be closed. Two facts combine to defeat any purely syntactic gate:[docs/limits.md]

1. *Open-type ambiguity.* A public reveal and a masked comparison-open are the same opcode; the `True`/`False` flag controls only a post-open correctness check, not privacy. Whether an open is safe depends on whether its operand is masked—a dataflow property present in the assembly but not determined by opcode identity alone.
2. *An unforbiddable channel.* A secret can be moved into a subtape's open through the same `call_arg` register channel the honest subtapes use.

Enumerating channels is therefore a losing game against the root ambiguity in (1). We scope layer one honestly as a *linter*: it rejects the five committed negative controls and catches gross or accidental leaks and regressions, but a PASS is explicitly not a non-leakage certificate. The runtime banners of the tools say so.[delivery_inspect.py; private_run.py]

4.3 An independent semantic bypass

On a different axis, a build can deliver $(\text{accept}_j, \text{payload}_j)$ privately to exactly the correct recipient and still compute the *wrong* secret function inside. Privacy of delivery says nothing about *which* value is delivered. Functional conformance binds source to specification, but only up to test coverage, not in general. This "correct recipient, wrong function" gap is orthogonal to the six leak channels and is why privacy alone is insufficient.[docs/limits.md]

4.4 Can the boundary be made a theorem?

The directive under which this work proceeds is explicit that we must not overstate. What Sections 4.1–4.2 establish is an empirical demonstration plus a structural argument, for *one* backend and the *studied* opcode-level approach. We do not assert a general impossibility. A stronger claim would be:

Conjecture 1 (dataflow-free syntactic certification is unsound)**.** *Fix the pinned MP-SPDZ Rep3 backend and the functionality $F =$ `threshold_SMC`. Call a checker* syntactic/dataflow-free *if it is a computable predicate on the compiled tape-manifest that is invariant under a transformation class T which preserves opcode identity and tape structure but may relabel an open's operand (substituting a verdict-derived register for a masked one) and may move a register between tapes via the `call_arg` channel. Then there exist source programs P_{safe} (delivers F privately) and P_{leak} (leaks a verdict) whose compiled manifests are T-related; hence any such checker C satisfies $C(P_{\text{safe}}) = C(P_{\text{leak}})$ and cannot both accept P_{safe} and reject P_{leak}. No dataflow-free syntactic checker is sound and complete for non-leakage on this backend.*

We state this as a conjecture, not a theorem, because three obligations are unmet:

Proof obligation 1. A formal model of the compiled manifest and a rigorous definition of the transformation class T—i.e., a precise formalization of "does not do dataflow" as invariance under operand relabeling and cross-tape `call_arg` moves.

Proof obligation 2. An explicit T-related pair $P_{\text{safe}}, P_{\text{leak}}$ with a proof that they are T-equivalent and that the honest build is a fixed point of the gate. We have empirical instances (the B3/B4 `reveal(False)` pairs), not a proof of T-equivalence.

Proof obligation 3. A protocol-level definition of "leak," to prove P_{leak} actually leaks and P_{safe} does not.

Obligation 3 is the crux: proving the impossibility rigorously requires the very simulation-based privacy definition whose absence is the boundary this paper is about. We therefore keep the asserted result empirical and mark Conjecture 1 **[OPEN: theorem: formalize T, exhibit a proven T-equivalent pair, and supply a protocol-level leak definition]**. Crucially, a *dataflow-aware* analysis is *not* covered by the conjecture and is not claimed impossible; Section 5 lists it as a route to a real guarantee.

5 The Hand-off: What Would Make It a Real Guarantee

The boundary is not that privacy is unknowable; it is that the evidence type changes. A real non-leakage guarantee needs two *separate* properties, and functional conformance supplies neither in general.[docs/limits.md]

Privacy of the executed circuit. A protocol-level Rep3 simulation argument (a human MPC specialist), or a formally-verified comparison primitive plus a dataflow analysis proving every open's operand is masked and no verdict-derived register flows into any open. This shows the executed functionality is computed privately.

Semantic source-to-spec binding. An argument that the circuit computes the *intended* belief-tracking function, not merely one that agrees with the oracle on the tested cases. Without this, privacy still permits the "correct recipient, wrong function" build of Section 4.3.

A Rep3 proof alone establishes the first and not the second; conformance testing gives partial, coverage-bounded evidence toward the second and nothing toward the first. Table 2 places each property with the actor that must supply it. The practical recommendation is that an artifact should represent this boundary in its own outputs—our tools print a linter caveat at runtime and the threat-model document leads its "not claimed" list with sound non-leakage by static inspection—so that a green check is never mistaken for a privacy proof.

6 The Adversarial Review Process

The results above were produced by a three-party loop: direction, implementation, and adversarial review, with a shared repository as the only mailbox.[docs/workflow.md] The reviewer is instructed to refute rather than assess, and to default to "refuted" when uncertain. Every critique is logged in an append-only review log, whether it survived or not; the log has 47 entries.[docs/review-log.md] We report the process as a contribution for two reasons. First, it has teeth: it caught a fatal specification bug (Section 3) and produced the entire bypass sequence of Section 4. Second, it is self-correcting in a way worth documenting—our own soundness premise that "memory is the only cross-tape channel" was asserted at one round (justifying the memory-channel rules) and refuted at the next by the `call_arg` channel, and both the assertion and its refutation remain in the log.[review-log.md R-45,R-47] Two models agreeing is weak evidence, for correlated reasons, so the process is a filter, not a substitute: the project's standing rule is that a human MPC specialist must sign off before any security claim ships.

7 Related Work

We position against five categories. We flag at the outset that the novelty statements below are **[OPEN: literature review: the project has not run a search reaching Google Scholar or Semantic Scholar]**, and that the paper's framing depends on completing it.

MPC frameworks and compilers. MP-SPDZ [2] and peers provide the substrate; to our current knowledge they do not offer application-level conformance against an independent oracle nor certify per-recipient delivery **[OPEN: confirm]**.

Knowledge-based security and quantitative information flow. The functionality is from PLAS 2012 [1], which simulated rather than implemented SMC belief tracking; we target its deterministic-query fragment **[OPEN: confirm no later work implements it with a conformance target]**.

Formal verification of MPC and secure compilation. Verified MPC compilers, type systems for information flow, and computational-soundness results are the correct heavy machinery our boundary hands off to; the negative result is positioned as "why an opcode-level syntactic shortcut fails," not as competing with dataflow-aware verification **[OPEN: cite the specific lines of work]**.

Conformance, differential, and metamorphic testing of cryptographic code. Test-vector conformance and differential testing of crypto libraries are the closest methodological neighbors; the transfer to an MPC *application* with an anti-echo interface and recomputed evidence is the claimed novelty **[OPEN: substantiate against that literature]**.

Static analysis and linting for information flow. Our negative result names a specific failure mode for opcode-level linters of MPC bytecode: opcode identity versus dataflow, and an honest-used register channel **[OPEN: position against existing IFC-linting work]**.

8 Threats to Validity

Construct. "Conformance" means agreement with an independent oracle on tested inputs; the oracle is a human transcription of one paper's deterministic-query fragment and carried a fatal bug until review caught it, so unreviewed transcription errors may remain, and no MPC expert has yet signed off. *Internal.* The linter's soundness for the memory channel rests on empirical facts about the pinned backend (the clean build uses no memory; a cross-tape register reference is a compile error); a different backend version could change opcode semantics or channels. The `tls_certs_present` flag is a weak proxy (files exist $\neq$ wire encrypted), a caveat we state.[ADVERSARY.md] *External.* Results are for one functionality, N=3, domain $\{0, 1, 2\}$, one backend, semi-honest honest-majority, and a single-host harness that is not an adversarially isolated deployment; generalization is **[OPEN: E4]**. *The boundary result is empirical, not a theorem* (Section 4.4). *Semantic gap.* Even perfect delivery privacy would not establish that the circuit computes the intended function beyond tested cases. *No performance claims* are made; the repository's timings measure a different, non-conforming circuit and are explicitly not to be cited.[README; docs/gap.md]

9 Limitations and Future Work

Persistent secret-shared state across invocations (Σ_T) is unimplemented and, under the current directive, unauthorized; it is future work, not a gap we fill here. The single highest-value next step is the literature review that gates the novelty framing (Section 7). The experiment most likely to change a conclusion is a genuine *dataflow-aware* delivery checker: if it closes the gap the linter cannot, it changes the boundary; if it fails, it upgrades the negative result from "syntactic gates fail" to "here is exactly where dataflow is needed." **[OPEN: dataflow checker: new implementation, requires authorization]** Attempting the impossibility theorem of Section 4.4, generalizing beyond the fixture to larger domains and probabilistic queries, and obtaining the protocol-level Rep3 argument from a specialist are the remaining directions.

10 Conclusion

Executable conformance against an independent oracle, backed by an anti-echo interface and recomputed evidence, is a rigorous and reusable discipline for MPC applications: it establishes

real functional correctness on tested inputs, catches gross information leaks, and makes a green check mean something. It also has a sharp, nameable boundary. Attempting to certify per-recipient non-leakage by static inspection of the compiled assembly fails against an author who controls the source, for structural reasons—a masked open and a raw reveal are the same opcode, and the secret can travel the same register channel the honest code uses—and that boundary is exactly where implementation-level evidence must hand off to a protocol-level privacy proof. Naming the boundary, rather than papering over it with a green check, is the contribution we most want to stand.

A Claim–evidence–limitation table

Table 3 lists the claims the paper makes, each with its repository evidence and the limitation stated alongside it.

B A minimal leak and the honest channel

The `reveal(False)` bypass (B3/B4) reconstructs a verdict publicly with the "correctness-check-disabled" flag, which the earlier gate wrongly treated as safe:

```
# spoofed EQZ(3)_63 subtape: no stdout, no privateoutput, no sink
w = sint.load_mem(4000) # verdict moved from main
w.reveal(False) # STILL a public open; False != private
```

B6 is different: the honest comparison subtapes receive their operands through the same register channel a leak would use, so it cannot be forbidden:

```
# main tape invokes a comparison subtape by passing registers directly:
# call_tape 12, 3, ci0, ... s6(3), s0(3) ... (no memory)
# the subtape receives them via call_arg -- the honest path.
```

References

[1] P. Mardziel, M. Hicks, J. Katz, and M. Srivatsa. Knowledge-Oriented Secure Multiparty Computation. In *PLAS*, 2012. **[OPEN: full citation and DOI/URL to be finalized]**.

[2] M. Keller. MP-SPDZ: A Versatile Framework for Multi-Party Computation. In *ACM CCS*, 2020. **[OPEN: confirm exact citation details]**.

ID	Mutation	Why it passed the then-current gate	Closed?	Committed control
B0	public `reveal()` in main, keep per-party print	stdout is a public channel; the oracle cannot tell private output from public open + private print	yes	`leaky.mpc`
B1	separate `@function_tape` public reveal + print	inspector reads only the main tape	yes	`subleak.mpc`
B2	name-spoofed `EQZ(spoof)` subtape: wrong-player delivery + file sink	masked allowlist keyed on tape *names*	yes	`namespoof.mpc`
B3	`reveal(False)` in an EQZ/LTZ-named subtape	gate treated `False` opens as safe (they are still public)	yes	`openfalse.mpc`
B4	`reveal(False)` in an *existing* subtape body via scalar `stms`/`ldms`	multiset pins tape names/counts, not bodies	yes	`openfalse.mpc`
B5	same, crossing the boundary with vectorized `vstms`/`vldms`	memory regex matched only scalar/gf2n	yes	`openfalse_vec.mpc`

Property	Evidence type	Supplied by
Functional correctness (tested)	conformance vs. independent oracle	this work
No gross delivery leak	static linter + strict transcript	this work
Evidence integrity	recomputed, SHA-bound provenance	this work
Executed-circuit privacy	Rep3 simulation or verified dataflow primitive	**[OPEN: expert / future]**
Semantic source-to-spec binding	correctness argument beyond test coverage	**[OPEN: future]**

Table 2: The hand-off. The top three rows are what implementation-level evidence establishes; the bottom two require a protocol-level argument this work does not provide.

Claim	Evidence	Limitation
C1 Circuit reproduces the oracle on an accept-and-reject fixture	`harness.py`, `oracle.py`, 4/4 named cases	per-tested-input, not general; one fixture is a regression vector
C2 Agreement across a 228-case adversarial sweep	`coverage.py`; `216 single + 12 carried = 228 pass`	fixed enumerable set at N=3, D=$\{0,1,2\}$
C3 Oracle is independent and transcribes all-outputs `tcheck`	`oracle.py`, R-13 fix, discriminating test	human transcription; no expert sign-off
C4 Interface forbids echo-cheating	`INTERFACE.md`, extra secret state	enforced by construction/review, not a machine proof
C5 Signed bound is $B < 2^{k-1}$; fixture safe	`circuit_spec.py`, R-22	general bound stated, not auto-checked
C6 Linter rejects five negative controls	`delivery_inspect.py`, five `.mpc` controls	a linter, not a non-leakage proof
C7 Runtime transcript fails closed	`strict_parse_party`, `test_private.py`	cannot see shares/-timing/network
C8 Evidence is typed, SHA-bound, recomputed	`validate_evidence.py`; CI `--recompute`	does not bind source to intended spec beyond tested cases
C9 Provenance is real (bound only in CI)	`repo_provenance()`, R-34	not a reproducibility guarantee on arbitrary machines
C10 Studied syntactic gate is unsound vs. a source-controlling author	R-41–R-47, `docs/limits.md`	empirical + structural, not a theorem; this backend
C11 Conformance binds source to spec only up to coverage	`docs/limits.md`, `gap.md`	motivates the semantic hand-off property
C12 The review loop is auditable and self-correcting	`review-log.md` R-45→R-47	process evidence, not completeness

Table 3: Claim, evidence, and limitation—grounded only in the repository at `305a3a8`.